

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

4: Data structures

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

1

- Basic data types and wrapper classes
 - Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
 - Designing your own data structures
 - The important stuff
 - Assignments
- 
- To be able to use existing (build in) data types in Java
 - To be able to understand OO philosophy behind data types in Java
 - To be able to utilize OO concepts in designing your own data structures

SIMPLE DATA TYPES

- Lower case number types, such as `int`, `double`, `float`, `byte` are raw numbers expressed as integer (of some word length), IEEE 754 standard, `byte` (or word of length 8 bit)
 - Those can be handled just as raw numbers, and are exchanged by value
- Capital case number types, such as `Int`, `Double`, `Float`, `Byte`, are Classes that carry the value of an integer, double, float or byte, adding some more methods, e.g.
 - `parseInt(String S)` – takes a string `S` and returns an `int` (if possible)
 - `longValue()` – returns the integer as a `long`
 - `rotateLeft(int i, int distance)` – rotate the bits to the left
- These are called wrapper class objects and are object forms of primitives (or simple values) that adds object semantics

WRAPPER CLASSES ON BASIC DATA TYPES

- Normally we just use basic data types as intended, such as byte, long etc.
 - `byte a = 35;`
 - `double y = 1.245;`
- But sometimes we need to take an input from the keyboard, using e.g. Scanner, and we then need to convert it into some special data type
- Or we need to be able to handle the data type as an object

Basic data type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

WRAPPER CLASSES ON BASIC DATA TYPES

- We can use the wrapper class object instance as if they were basic data types, e.g.
 - Integer x = 5; Double y = 1.234;
- Printing is just as easy
 - System.out.println(x);
 - System.out.println("Y has the value "+y);
- It is also doable to access the value via proper methods
 - System.out.println(x.intValue());
 - System.out.println(y.doubleValue());

WRAPPER CLASSES ON BASIC DATA TYPES

- But the real strength is in most other related methods e.g. `toString()`
 - `Double x = 1.234; String strX = x.toString();`
- Or the other way around, if we have a string and need the data
 - `double f = Double.parseDouble(strX);`
- In C we would have had to locate an appropriate function, but here we get the method with the wrapper data type that we wish to use
 - `int y = Integer.parseInt(strX);`
 - `byte b = Byte.parseByte(strX);`
- In many cases, we even have more options than just to pass a string, which are suitable for the appropriate data type
- If something fails, an exception is thrown (more about this later)

WRAPPER CLASSES EXAMPLES

- Notice, double is default in Java, so if we wish to use float (lower precision), then, in Test2, we must cast 2.766 to float, e.g.
 - `float f2 = (float) 2.766;`

```
public class Test {  
    public static void main(String[] args) {  
        Integer myInt = 5;  
        Double myDouble = 5.99;  
        Character myChar = 'A';  
        System.out.println(myInt);  
        System.out.println(myDouble);  
        System.out.println(myChar);  
    }  
}
```

```
public class Test2 {  
    public static void main(String[] args) {  
        String str = "1.234";  
        double f = Double.parseDouble(str);  
        System.out.println("Double is "+f);  
        Double f2 = 2.766;  
        double f3 = f+f2;  
        System.out.println("Result is "+f3);  
    }  
}
```



WRAPPER CLASSES EXAMPLES

7

```
1 package lecture4;
2
3 public class Temperature {
4     private double value; // primitive wrapped inside an object
5
6     public Temperature(double value) {
7         this.value = value;
8     }
9
10    public double getValue() {
11        return value;
12    }
13
14    public double toFahrenheit() {
15        return value * 9/5 + 32;
16    }
17
18    public double toKelvin() {
19        return value + 273.15;
20    }
21
22    @Override
23    public String toString() {
24        return value + " °C";
25    }
26 }
```

```
1 package lecture4;
2
3 public class WrapperDemo {
4     public static void main(String[] args) {
5         Temperature t = new Temperature(25.0);
6
7         System.out.println(t); // 25.0 °C
8         System.out.println(t.toFahrenheit()); // 77.0 °F
9         System.out.println(t.toKelvin()); // 298.15 K
10    }
11 }
```

What happens if the **this** keyword is removed? And why is that?

AGENDA

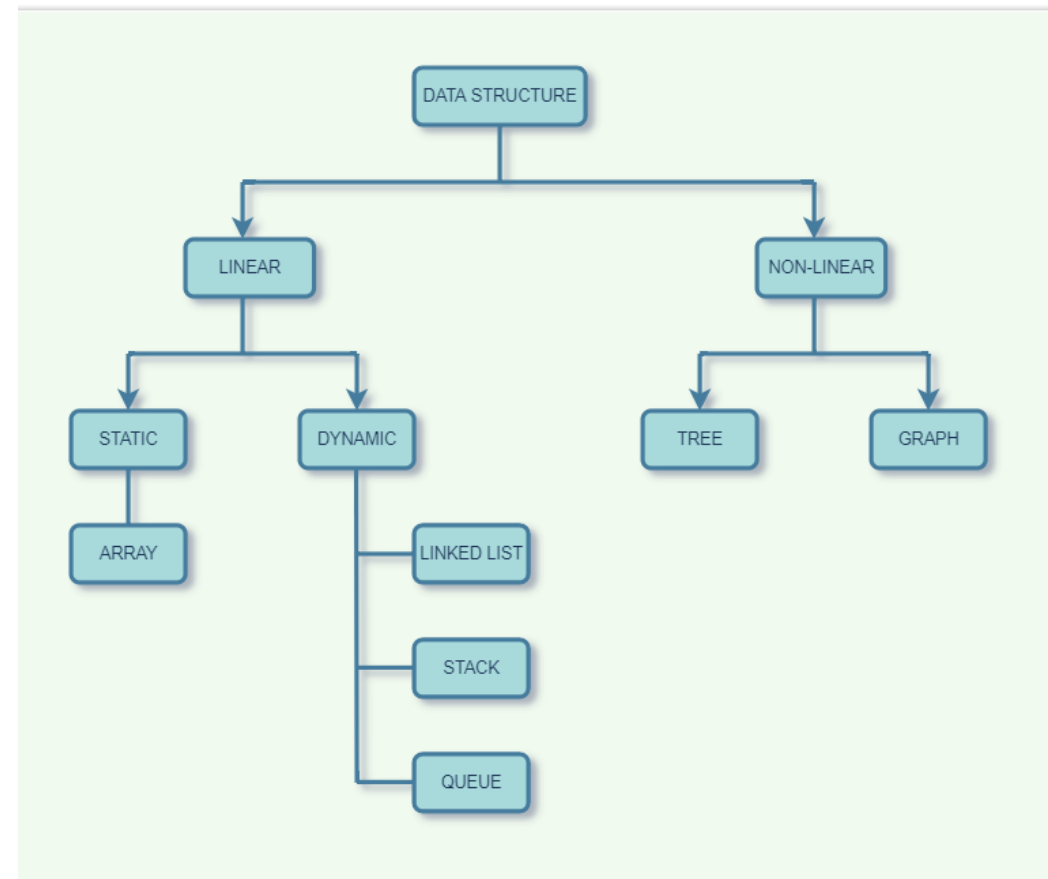
- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

DATA STRUCTURES

- As in imperative languages we have a whole range of different data types here

- Arrays
- Double arrays
- Strings
- Linked lists
- Stack
- Queue
- Hashtable

- And many more



ARRAYS

- For simple data types, arrays are defined and used very simply

```
int[] priceOfPen= new int[5];  
priceOfPen[3]=5;
```

This gives [0, 0, 0, 0, 0]

This gives [Null, Null, Null, Null, Null]

- For complex (or just our own classes) we need to initialise these manually. The array itself is initialized but the objects inside it do not yet exist

```
Coordinate[] myCoordinates = new Coordinate[5];  
myCoordinates[0]=new Coordinate(3,4);
```

- This we need to do for each of them, so consider using e.g. a *for* loop

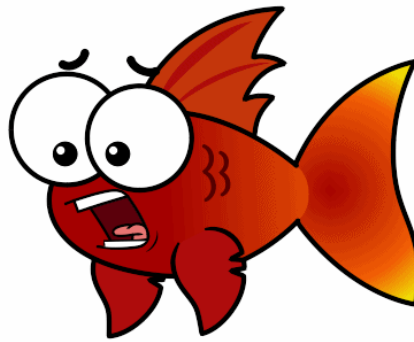
ARRAY EXAMPLE CODE

- If we just print cars then we get the object reference
 - Yes, even an array is an object
- The array class have several attributes, e.g. length, which indicates how many elements exists
- This is very useful for going through the array

```
public class ArrayTests {  
  
    public static void main(String[] args){  
        String[] cars = {"Volvo", "BMW", "Ford",  
                          "Mazda"};  
        System.out.println(cars);  
        for (int i=0; i<cars.length;i++) {  
            System.out.println("Car "+i+" "+cars[i]);  
        }  
    }  
}
```

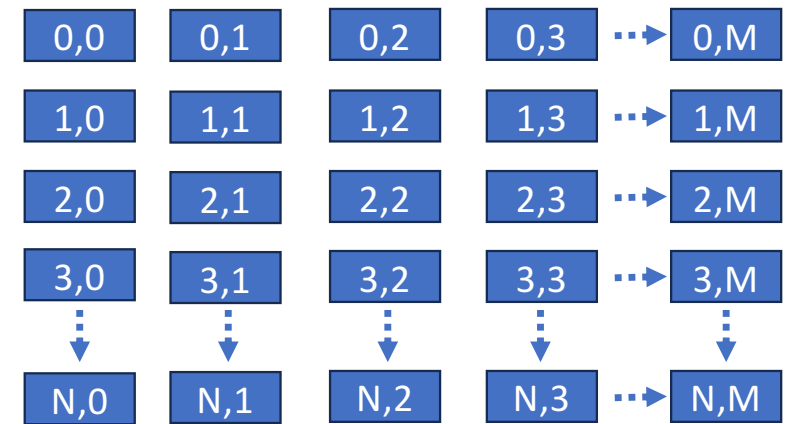
BREAK – PAUSE(FISK)

12



MULTI-DIMENSIONAL ARRAYS

- Multi-dimensional arrays can be formed as simply array(s) of array(s)
- This is quite useful in many situations
 - Matrices
 - Storing data as a tabular form
 - Data with fast access
 - Maps
- Jagged arrays
 - Arrays of arrays
 - Each row can have a different size
 - Flexible (can model irregular data)
 - Saves (potentially) memory



```
data_type[1st dimension][2nd dimension][...][Nth dimension]
array_name = new data_type[size1][size2]....[sizeN];
```

```
int[][] twoD_arr = new int[10][20];
int[][][] threeD_arr = new int[10][20][30];
```

```
MyClass[][] myClass = new MyClass[10][10];
myClass[0][0]=new MyClass("foo");
```

MULTI-DIMENSIONAL ARRAYS

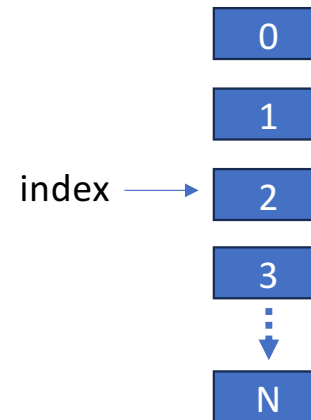
- We can also initialize the arrays if needed
- Notice, that each element `myNumbers[x]` is also an array object
- Same concept for higher dimension arrays

```
public class ArrayTest2 {  
    public static void main(String[] args) {  
        int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };  
        for (int i = 0; i < myNumbers.length; ++i) {  
            for(int j = 0; j < myNumbers[i].length; ++j)  
            {  
                System.out.println(myNumbers[i][j]);  
            }  
        }  
    }  
}
```

```
[  
  [1, 2, 3, 4],  
  [5, 6, 7]  
]
```

ARRAYLIST (LIST)

- Basically, ArrayList is a resizable array with some additional features
- It implements interface List, which allows us to use stuff like
 - `Collection.sort(myList)`
 - `Collections.sort(myList, Collections.reverseOrder());`
 - given that `myList` implements a List interface, e.g. is an ArrayList
- It requires importing of the feature
 - `import java.util.ArrayList;`



ArrayList

+add()
+get()
+set()
+remove ()
+clear()
+size()
... and many more

```
ArrayList<String> cars = new ArrayList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");  
  
cars.get(0);  
cars.set(0, "Opel");  
  
Collections.sort(cars);
```

ARRAYLIST (LIST) EXAMPLE

- Here is a small example on how to use ArrayList
- Notice how we loop through the list
- ArrayList is extremely versatile as it can be dynamically resized as needed
- Methods are provided to add and remove array items, which makes this data type very flexible and easy to operate with

```
1 package lecture4;
2
3 import java.util.ArrayList;
4
5 public class ArrayListExample {
6     public static void main(String[] args){
7         ArrayList<String> cars = new ArrayList<String>();
8         cars.add("Volvo");
9         cars.add("BMW");
10        cars.add("Ford");
11        cars.add("Mazda");
12
13        for (String i : cars) {
14            System.out.println(i);
15        }
16    }
17 }
```

AGENDA

17

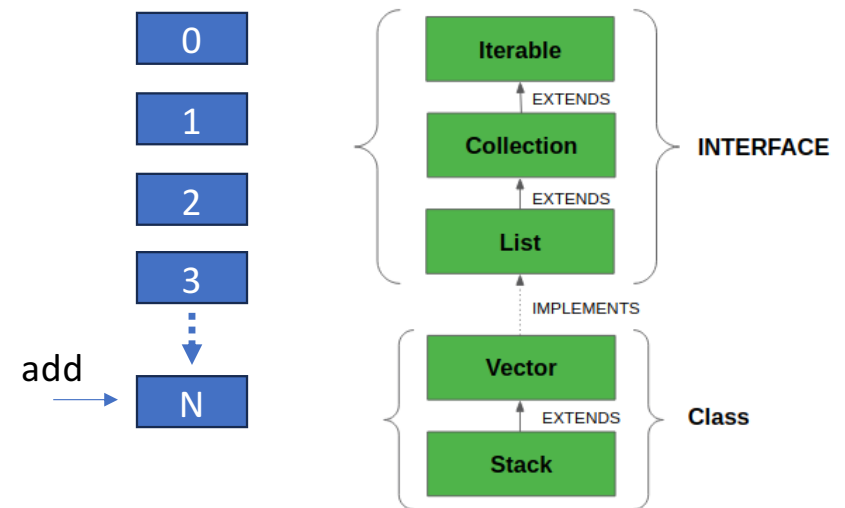
- Basic data types and wrapper classes
- Data structures
 - Arrays
 - **Vector**
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

VECTOR

18

- A Vector is a flexible array, very similar to ArrayList, as both are dynamic, indexed, resizable arrays that implement the List interface
- It is a legacy class – but unlike ArrayList it is synchronized
 - It has some additional methods not available in ArrayList
 - However, is also a bit legacy

```
Vector<E> v = new Vector<E>();  
Vector<E> v = new Vector<E>(int size);  
Vector<E> v = new Vector<E>(int size, int incr);  
Vector<E> v = new Vector<E>(Collection c);
```



```
Vector<Integer> v = new Vector<Integer>(n);  
v.add(7);  
v.add(9);  
v.add(23);  
v.ad(4);  
  
v.remove(1);  
System.out.println("Vector: "+v);
```

LINKEDLIST

19

- Another structure is the LinkedList, where we do not need to write code to use linked lists



```
LinkedList<String> cars = new LinkedList<String>();  
cars.add("Volvo");  
cars.add("BMW");  
cars.add("Ford");
```

LinkedList

```
+addFirst()  
+addLast()  
+removeFirst()  
+removeLast()  
+getFirst()  
+getLast()  
... and many more
```

- We still need to think complexity when using this datatype – the code behind the class is not magically efficient
- Difference btw. LinkedList and ArrayList?
 - Efficiency between insert, delete and random-access operations
 - ArrayList is based on a contiguous memory approach

AGENDA

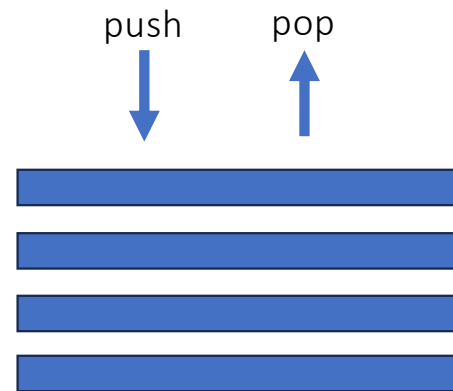
20

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

STACK

21

- A stack is a linear and simple data structure that follows the LIFO principle
 - Last In, First Out (LIFO)
- When to use this data type?
 - Whenever you have a case where you want to have a LIFO type of interaction, then you may need a stack
 - Shines for problems where backtracking, reversal or nesting structure handling are needed
 - The ctrl-z is a good example



Stack

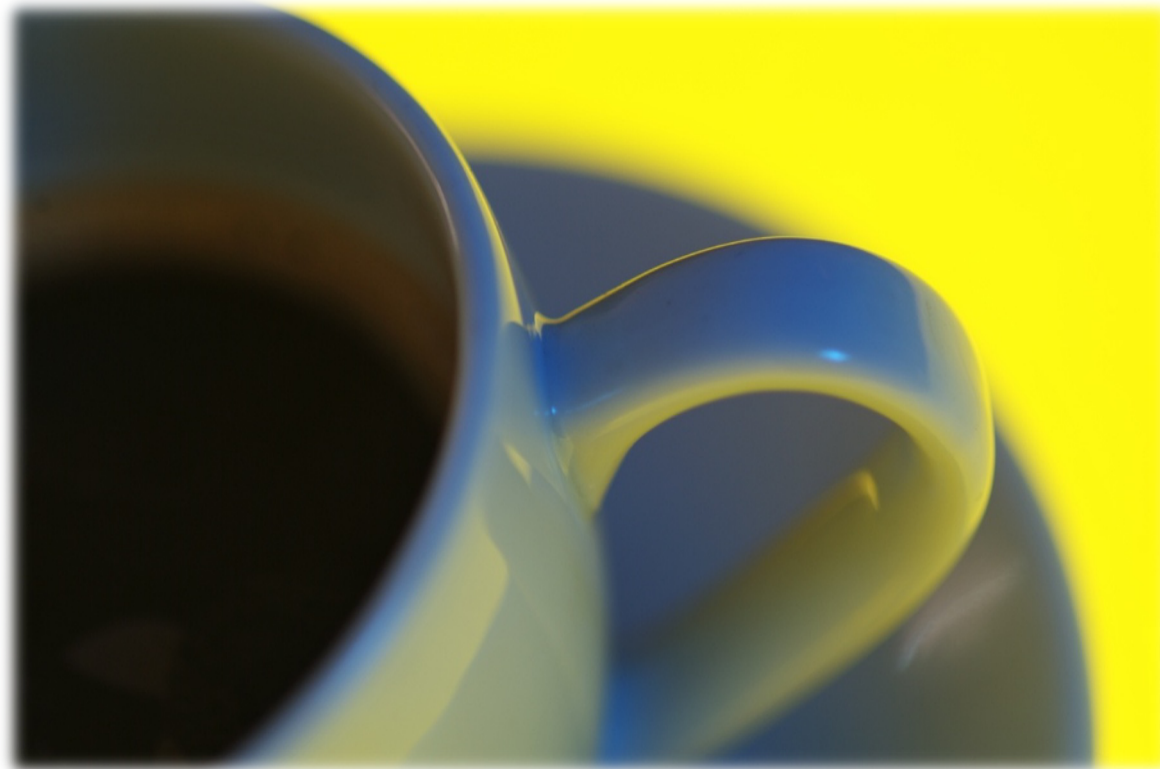
+push()
+pop()
+peek()
+search()
+capacity()
.... And many other

```
Stack stack = new Stack();
stack.push("4");
stack.push("All");

while(!stack.isEmpty()) {
    System.out.println(stack.pop());
}
```

"BIKER BREAK!"

22



Jan H. Mikkelsen, OOADI4, CoCT, 2026

AGENDA

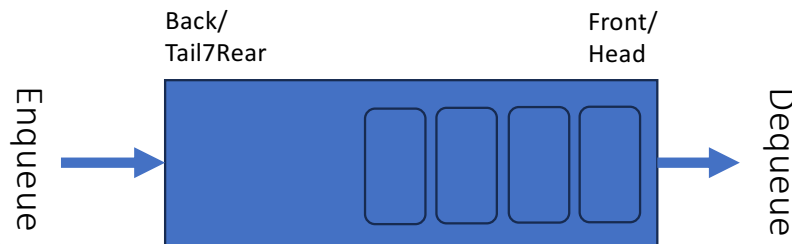
23

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

QUEUE

24

- Also a simple datatype but one that uses FIFO
 - First In, First Out
- Is actually a java interface
 - LinkedList
 - PriorityQueue
 - And some other
- A Queue when the order of processing matters and the oldest item should be handled next
 - print jobs



Queue (interface)

```
+add()  
+remove()  
+sort()  
+peek()  
+size()  
+clear()  
... and many more
```

```
Queue<Obj> queue = new PriorityQueue<Obj> ();  
  
queue.add("apple");  
queue.add("banana");  
queue.add("cherry");  
  
String front = queue.remove();  
String peeked = queue.peek();
```

AGENDA

25

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

HASHTABLE / HASHMAP

- Hashtables offers efficient $\langle \text{index}, \text{value} \rangle$ access to data
 - Index is the hash value of the key
 - The hash can be calculated in many ways
e.g. $\text{index} = \text{key} \bmod \text{noOfStorageElements}$
- Other types also exists, e.g.
 - HashMap
 - LinkedHashMap
 - ConcurrentHashMap
 - HashSet

Queue (interface)

```
+put()
+remove()
+get()
+isEmpty()
+keys()
+rehash()
... and many more
```

```
Hashtable<Integer, String>
    hm = new Hashtable<Integer, String>();

hm.put(1, "Geeks");
hm.put(12, "forGeeks");
hm.put(15, "A computer");
hm.put(3, "Portal");

Integer n = numbers.get("forGeeks");
```

HASHMAP EXAMPEL

- Printing the object prints all <key,value> pairs
- If we do not find an object with a key, we get a null
 - A very common source of errors!!
 - *.get("Denmark") returns Null

```
import java.util.HashMap;

public class TestHashMap {
    public static void main(String[] args) {
        HashMap<String, String> capitalCities = new HashMap<String, String>();
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("USA", "Washington DC");
        System.out.println(capitalCities);
        System.out.println(capitalCities.get("England"));
        System.out.println(capitalCities.get("Denmark"));
    }
}
```

```
{USA=Washington DC, Norway=Oslo, England=London, Germany=Berlin}
London
null
```

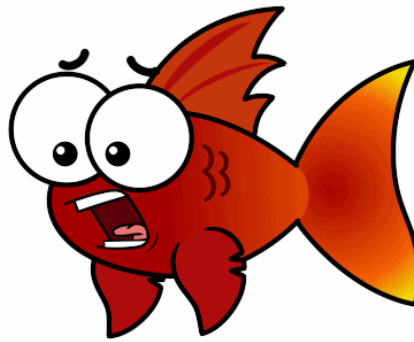
HASHMAP EXAMPEL

- *.get("Denmark") returns Null
- The key is not present in the map
→ normal situation, but you need to handle it
- The key *is* present, but the value stored is in fact null
→ this is allowed!
- You cannot tell these two cases apart by simply checking get()

```
import java.util.HashMap;

public class TestHashMap {
    public static void main(String[] args) {
        HashMap<String, String> capitalCities = new HashMap<String, String>();
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("USA", "Washington DC");
        System.out.println(capitalCities);
        System.out.println(capitalCities.get("England"));
        System.out.println(capitalCities.get("Denmark"));
    }
}
```

BREAK – PAUSE(FISK)



AGENDA

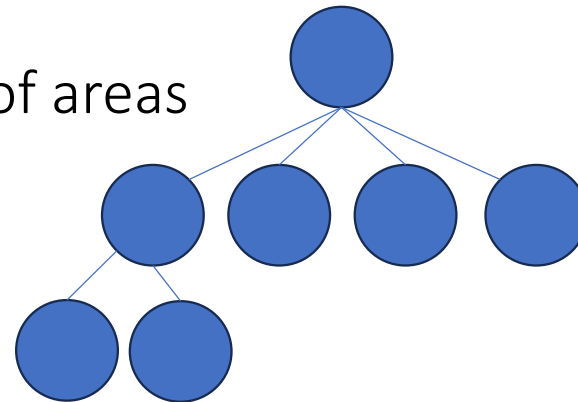
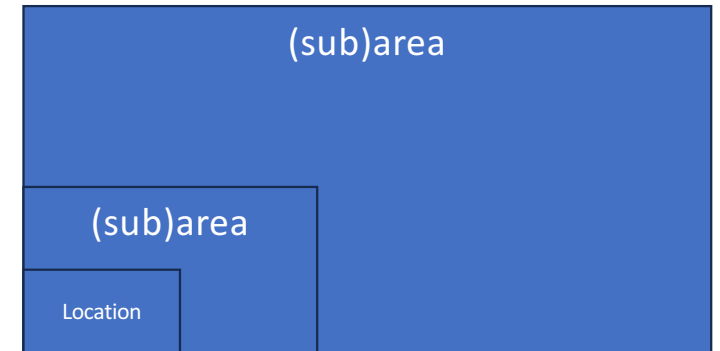
30

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

THREE STRUCTURES

31

- Problem: we wish to search for a specific location
 - Linear search through a list of Locations
- Define a datatype Area which holds
 - Information about (sub)Areas, e.g. what types exists beneath this
 - Methods to check if a location is in this range
- Then we may create an n-ary tree of areas and locations at the bottom
 - Yields more efficient searches like
 - What tourist attractions are nearby me
 - Where is the nearest railway station



THREE STRUCTURES

- What can we do with data tree structures?
 - Insert, add, remove, ...
 - Search, update, ...
 - Checks e.g. `isLocationInRange()`
- So, we add and implement these methods
 - Keeping all household data internal
 - Counters, pointers, objects, hashtables, whatever we use ...
 - Additionally, we may need nice prints
 - If we just print our tree, we get something like `MyTree@5ccd43c2` -> what does this mean?
 - This is an object reference – Most useful to check our object is not null
 - So, we may wish to add a `printNice()` method or similar that prints the information we want shown

THREE STRUCTURES

- The base is a model of a node, which
 - Has some data
 - Has children (for binary, left and right children)
 - Constructor we set the data
 - Maybe we also wish a nodeID and some other metadata
- Tree code is centred around the TLNode class

```
class TLNode
{
    int data;
    TLNode left, right;

    TLNode(int d){
        data=d;
    }
}
```

```
public class BinaryTree{
    static void preorder(TLNode r) {
        if(r==null)
            return;
        System.out.print(r.data+" ");
        preorder(r.left);
        preorder(r.right);
    }
    static void inorder(TLNode r){
        if(r==null)
            return;
        inorder(r.left);
        System.out.print(r.data+" ");
        inorder(r.right);
    }
    static void postorder(TLNode r)
    {
        if(r==null)
            return;
        postorder(r.left);
        postorder(r.right);
        System.out.print(r.data+" ");
    }
}
```

HEAP

34

- Useful for e.g.
 - Applications with priorities
 - Scheduling algorithms
 - Caching
- Code implements a complete max-heap data structure using an array and demonstrates fundamental heap operations
- Min heap and max heap
 - $O(1)$ for min or max elements

```
class Heap{
    int heapSize;
    void build_max_heap(int[] a)    {
        heapSize=a.length;
        for(int i=(heapSize/2);i>=0;i--)
            max_heapify(a,i);
    }
    void max_heapify(int[] a,int i)    {
        int l=2*i+1;
        int r=2*i+2;
        int largest=i;
        if(l<heapSize && a[l]>a[largest])
            largest=l;
        if(r<heapSize && a[r]>a[largest])
            largest=r;
        if(largest!=i){
            int t=a[i];
            a[i]=a[largest];
            a[largest]=t;
            max_heapify(a,largest);
        }
    }
}
```

```
int extract_max(int[] a){
    if(heapSize<0)
        System.out.println("underflow");
    int max=a[0];
    a[0]=a[heapSize-1];
    heapSize--;
    max_heapify(a,0);
    return max;
}

void increase_key(int[] a,int i,int key){
    if(key<a[i])
        System.out.println("error");
    a[i]=key;
    while(i>=0 && a[(i-1)/2]<a[i]) {
        int t=a[(i-1)/2];
        a[(i-1)/2]=a[i];
        a[i]=t;
        i=(i-1)/2;
    }
}

void print_heap(int a[]){
    for(int i=0;i<heapSize;i++)
        System.out.println(a[i]+" ");
}
```

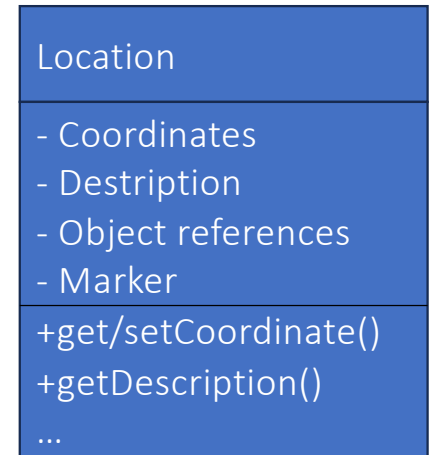
- As mentioned, Heap-structures excel at managing priority queues
- A max-heap ensures that “important” tasks are always first to be processed
 - CPU task scheduling
 - Print job prioritization
 - Event handling in GUIs
 - Network packet scheduling
- Other examples relate to constantly receiving a data stream and wanting to keep, for instance, a list of the top-10 largest values thus far
 - Top-10 high score in online game
 - Most popular hashtags today
 - Highest speed-trap measurements during a Formula One race

AGENDA

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

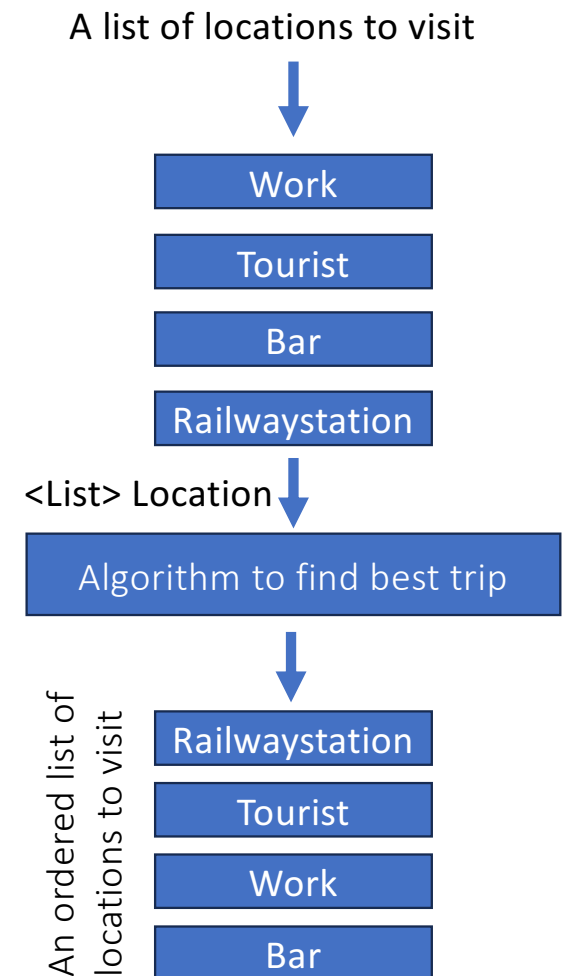
CUSTOM DATATYPE STRUCTURES

- Data types are not physical objects, but we treat (or can at least) them as if they are in OO
- Example: what characterizes a Location?
 - Coordinates <x, y> or <long, latt> or something else
 - A description for humans to relate (and machines)
 - A set of references to objects on this location
 - A marker how to show this graphically on a map
 - ... and probably more/other
- Maybe we have a special Location
 - Tourist attraction, which is located at a location
 - Transport point (e.g. railway station)
 - And ton of other options ...



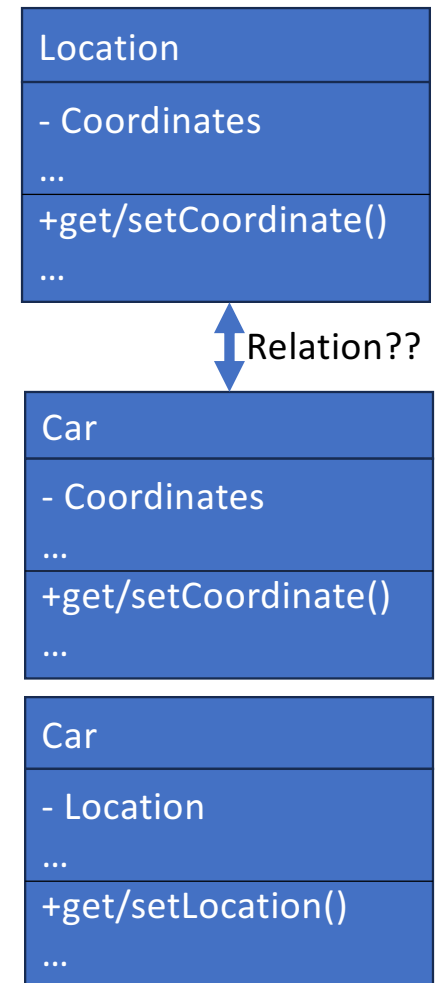
CUSTOM DATATYPE STRUCTURES

- Using inheritance, we may treat all tourist attractions as locations, as well as other location types
- Imagine a list of locations that I need to visit for various reasons
 - My job is on that location (work location)
 - I want to see the location (tourist attraction)
 - I need to meet someone there (a bar)
 - Etc. etc.
- I need now to create an algorithm that gives me the optimal (least time, least fuel, etc.) order of visits
 - Consequently, it is useful to treat all objects as Locations and not as specialized types
 - When I go to the location, it is useful to treat the object as specialized location



CUSTOM DATATYPE STRUCTURES

- There is a ton of different things you can do with custom datatype structure
 - But if we start to reuse (copy/paste) code elsewhere, then be aware
 - Ideally, we have specialized code in one, and only one, place
- Associations may also be a type of relations, e.g.
 - A car has a Location (as it is a physical object)



AGENDA

40

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments

THE IMPORTANT STUFF

41

- Primitive vs wrapper types
 - Understand memory (primitives -> stack, objects -> heap), behaviour, and when wrappers are needed
- Know your data structures
 - Each has specific strengths for specific access patterns
- OO principles drive Java's type system
 - Encapsulation, abstraction, and interfaces matter
- Data structures are behaviour + rules, not just storage
- Designing your own structure requires defining behaviour, choosing representation, and enforcing invariants

AGENDA

42

- Basic data types and wrapper classes
- Data structures
 - Arrays
 - Vector
 - Stack
 - Queue
 - Hash tables
 - Heap
- Designing your own data structures
- The important stuff
- Assignments



ASSIGNMENTS

- 1) Create an array with 100 bicycles – each with individual characteristics. Use the Random class to generate some random attribute values, see <https://www.geeksforgeeks.org/generating-random-numbers-in-java/>
Add an attribute name to the bicycle class, and code a namegenerator, such that you get a random and unique name for each bicycle, e.g. “myBicycle_1”
- 2) Place the 100 bicycles in a LinkedList, Stack and PriorityQueue and do the following
 - a. Add them one-by-one to a stack. Then decrease the speed for each bicycle by 25% as you take them off the stack
 - b. Add them one-by-one to a PriorityQueue. Then increase speed for each bicycle by 25% in the same order as you inserted the bicycles
 - c. Structure the bicycles in a LinkedList, then remove all bicycles that drives less than 10 km/h
- 3) Use a hashmap to organize the remaining bicycles and add a possibility to input a name from the user. Use the inputted name to search and find the hashmapped bicycle and print its info. Repeat this until the user types “exit”. To get user input you can use the Scanner class, see e.g. https://www.w3schools.com/java/java_user_input.asp